Matrix multiplication with many threads in Java

Pablo Castillo Garcerá
Big Data, Individual Assignment 3, 2026-2027
GitHub: https://github.com/Pablo5250/matrix-multiplication (folder assignment3)

Abstract

In this work I study how the number of threads changes the time of a dense matrix multiplication in Java. I take the blocked method from Assignment 2 as the sequential version, and I write a parallel version where several threads share the rows of the result. Both versions run the same code, so the only difference is the threads. I test matrices from 256 x 256 to 8192 x 8192 with 1 to 32 threads on a CPU with 12 cores, and I measure time, memory, speedup and efficiency. With big matrices, 12 threads made the multiplication about 7 to 8 times faster and 24 threads about 10 times faster. With small matrices of 256 x 256 the best result was only 2.7 times faster, and with more than 12 threads it got slower again. The conclusions are only valid for this computer, this Java version and this method.

1. Introduction and objective

In the first two assignments all my programs ran on only one core of the CPU. A core is one of the units inside the CPU that run instructions. My CPU, an AMD Ryzen 9 5900X, has 12 cores, so 11 of them did nothing while my programs were running. In Assignment 2 the fastest method, blocking, needed about 100 seconds to multiply two matrices of 8192 x 8192. In this assignment I want to use all the cores at the same time.

My research question is: how does the number of threads change the time of the multiplication, and when does it really help to run it in parallel? The factor I change is the number of threads, and I repeat the test with several matrix sizes. The outcome I measure is the time, and from the time I calculate the speedup and the efficiency (both are explained in section 2). I also measure the memory.

Before measuring, I expected three things. First, with big matrices the time should go down almost in proportion to the threads up to 12, because each core gets its own part of the work. Second, after 12 threads the gain should be small, because there are no more real cores. Third, with small matrices I expected the parallel version to help very little or even be slower, because creating the threads also costs time and the work itself is very short.

2. Background

A thread is a list of instructions that the computer runs on its own. A normal Java program runs my code in one thread. If I create more threads, the operating system can put each one on a different core, and then they work at the same time. This is called parallel execution. All the threads of a program share its memory, so all of them can read the same matrices [1]. The numbers in square brackets, like [1], are the sources I used. The full list is at the end of the report.

My CPU has 12 physical cores, but Windows sees 24 logical processors. The reason is that each core can run two threads at the same time. AMD calls this simultaneous multithreading (SMT). The two threads share the parts of the same core, so the second thread does not make the core two times faster. When a program has more threads than logical processors, for example 32, the operating system has to switch between them all the time. This is called oversubscription [3].

The 12 cores are also split in two groups of 6. Each group has its own L3 cache of 32 MB [4]. A cache is a small and very fast memory inside the CPU. If the numbers that the program needs are already in the cache, the

core does not have to wait for the RAM, which is much slower. Each core also has its own small caches, L1 (32 KB) and L2 (512 KB). This matters here because more threads also means more caches working at the same time.

Threads can also cause errors. When two threads write in the same place at the same time, the result can be wrong. This error is called a race condition, and it is hard to find because it does not happen every time [1]. That is why the assignment asks to check that the access to shared data is safe.

To compare the versions I use the two measures of the assignment. Tseq(n) is the time of the sequential version for matrices of size n, and Tp(n) is the time of the parallel version with p threads for the same n. The speedup is S(p) = Tseq(n) / Tp(n), and it says how many times faster the parallel version is. The efficiency is E(p) = S(p) / p, and it says how much of each thread is really used. For example, if 12 threads make the program 9 times faster, the efficiency is 9 / 12 = 0.75. A speedup bigger than the number of threads is called superlinear, and it is not normal, so it needs an explanation.

Amdahl's law explains why the speedup has a limit. If one part of a program cannot be split between threads, that part takes the same time with any number of threads, and with many threads it ends up deciding the total time [2]. In my program one example is the creation of the empty result matrix C, which Java does with only one thread.

3. Methodology

3.1 The two versions

The sequential version is the blocked method of Assignment 2. It multiplies the matrices in tiles of 64 x 64 numbers. A tile uses 32 KB, the size of the L1 cache, so its numbers stay in the cache while they are used. I made only one change: the work of each band of 64 rows of C is now its own method. Section 5 explains why this change was necessary.

The parallel version uses plain Java threads, without any library. First of all, it cuts the rows of C into chunks, 4 chunks for each thread (for example 48 chunks with 12 threads). Second, it starts the threads. Each thread takes a number from a shared counter, like people at the market taking a ticket, and it multiplies the rows of that chunk. When it finishes, it takes the next free number, until there are no chunks left. So a thread that runs slower simply takes fewer chunks. Third, the main thread waits until all the threads finish (this is called join) and returns C. Each thread runs exactly the same code as the sequential version, only on its rows.

The threads do not need locks to be safe. A and B are only read. Each row of C is in only one chunk, so two threads never write the same number. The shared counter is an AtomicInteger, a Java class that gives each number to only one thread, even when two threads ask at the same time [1]. Each value c[i][j] adds its products in the same order (k = 0, 1, ..., n-1) in both versions, so the result is exactly the same, bit by bit, with any number of threads.

I did not do the optional part about vectorisation.

3.2 Inputs and environment

Both versions multiply the same matrices. They are made in Java with random numbers of type double (64 bits) between 0 and 1, with java.util.Random and the seeds 42 + n for A and 4242 + n for B. These are the same matrices as in Assignment 2. A seed is the starting number of the random generator, so with the same seed every run gets exactly the same matrices.

The computer is an AMD Ryzen 9 5900X with 12 physical cores and 24 logical processors, 32 GB of RAM and Windows 11. I used OpenJDK 25.0.1. I did not set any affinity, which means that I did not tell Windows on which core each thread must run. Windows decides it.

The assignment also asks to control the threads that are not mine, so that the thread count I report is the real one. My code does not use any library. However, Java has its own threads. The JIT compiler turns the Java code into fast machine code while the program runs, and the garbage collector (GC) frees the memory that is not used any more. By default the GC of Java uses several threads. I limited it to one thread with the options -XX:ParallelGCThreads=1 and -XX:ConcGCThreads=1 [5], and I let Java use up to 8 GB of memory (-Xmx8g). The JIT threads work mostly during the warm-up runs, which I do not count.

3.3 Correctness checks

Before measuring I checked the code with 87 tests. Some use small matrices that I multiplied by hand (2 x 2 and 3 x 3). Others use the identity matrix (ones on the diagonal and zeros in the rest), the zero matrix, sizes that are not a multiple of 64 (70 and 130), and more threads than rows, for example a 2 x 2 matrix with 32 threads. The sequential version and every thread count from 1 to 32 give exactly the same numbers as the basic i-j-k method of Assignment 1, which I checked against NumPy in that assignment. To look for race conditions, I repeated the multiplication 200 times with 2, 12, 24 and 32 threads, and the result was always the same.

In the big runs I also compared the sum of all the values of C with the sequential version. It was exactly the same in all the 54 parallel configurations and in all their runs.

3.4 How I measured

The timer (System.nanoTime) measures the whole multiplication: creating the result matrix, starting the threads, the work and waiting for the threads. Creating A and B is not measured. After each run the program adds all the values of C, so Java cannot skip the work.

Each configuration (one size, one version and one thread count) runs in a new Java process. First it does some warm-up runs that I do not count, because the JIT compiler needs time to optimise the code, and then 5 measured runs. Small sizes finish in a few milliseconds, so they need more warm-up runs: 100 for n = 256, 30 for n = 512, 10 for n = 1024 and 3 for the rest. For the time I report the median, which is the middle value of the 5 runs, and the IQR (interquartile range), which shows how different the runs were. Speedup and efficiency are calculated with the medians.

The sizes are 256, 512, 1024, 2048, 4096 and 8192. The thread counts are 1, 2, 4, 6, 8, 12, 16, 24 and 32. I chose them for a reason: 6 threads fill one group of cores, 12 fill all the physical cores, 24 fill all the logical processors and 32 is oversubscription.

Before starting I also decided some limits: 20 minutes and 8 GB of memory for each configuration. If a configuration was predicted to need more time, I skipped it and wrote it down. For memory I give the peak working set, which is the maximum RAM used by the Java process. I read it with psutil, a Python library that reads information about the programs that are running, every 10 ms. psutil also gives the CPU time of the process. If I divide it by the real time on the clock, I get how many cores the process used on average.

4. Results

All the 60 configurations finished inside the limits, also the biggest size, n = 8192. The slowest one was the sequential version at n = 8192: 112 s for each multiplication and 15 minutes for the whole configuration with the warm-up runs. So the largest size I tested and the largest size that worked are the same, n = 8192.

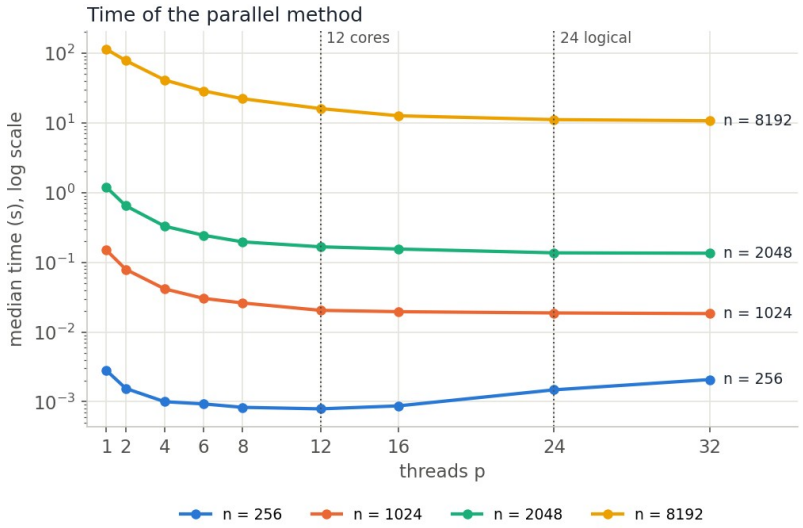


Figure 1. Median time of the parallel version for four sizes. The y axis uses a logarithmic scale, where each step is 10 times more.

Table 1. Median time of 5 runs. Seq. is the sequential version, with no threads, and p is the number of threads of the parallel version.

n	Seq.	p = 1	p = 2	p = 4	p = 6	p = 8	p = 12	p = 16	p = 24	p = 32
256	2.12 ms	2.82 ms	1.55 ms	1 ms	0.93 ms	0.831 ms	0.79 ms	0.871 ms	1.48 ms	2.08 ms
512	17.6 ms	20.9 ms	10.8 ms	5.63 ms	4.26 ms	3.75 ms	2.75 ms	2.72 ms	2.85 ms	3.2 ms
1024	143 ms	149 ms	79 ms	41.4 ms	30.3 ms	26.1 ms	20.4 ms	19.5 ms	18.7 ms	18.3 ms
2048	1.32 s	1.19 s	644 ms	328 ms	243 ms	196 ms	166 ms	155 ms	136 ms	135 ms
4096	11.4 s	10.8 s	8.15 s	4.46 s	3.12 s	2.38 s	1.69 s	1.38 s	1.2 s	1.16 s
8192	112 s	113 s	77.8 s	40.9 s	28.6 s	22.1 s	15.9 s	12.6 s	11.1 s	10.7 s

Figure 1 and Table 1 show that with big matrices the time goes down very fast with the first threads. At n = 8192 the sequential version needs 112 s, 12 threads need 15.9 s and 24 threads 11.1 s. After 24 threads the line is almost flat. With n = 256 the line goes down until 12 threads and then goes up again.

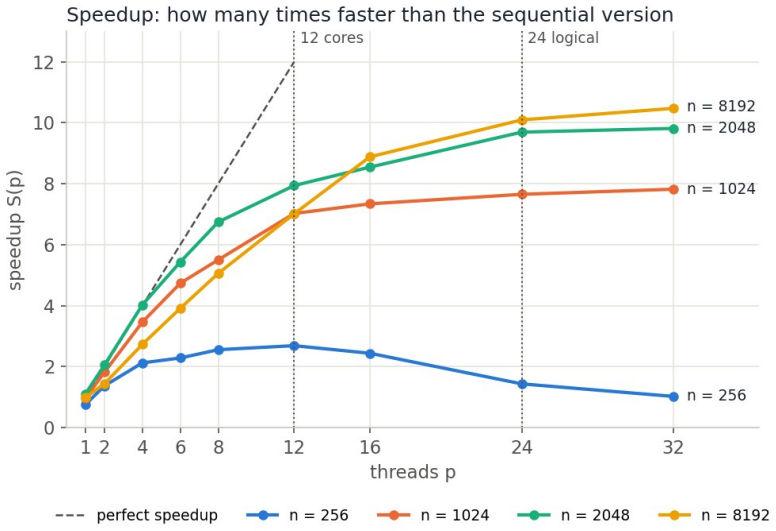


Figure 2. Speedup of the parallel version. The dashed line is the perfect speedup, where p threads are p times faster.

Table 2. Speedup S(p) = Tseq / Tp, with the medians of Table 1.

n	p = 1	p = 2	p = 4	p = 6	p = 8	p = 12	p = 16	p = 24	p = 32
256	0.75	1.37	2.12	2.28	2.56	2.69	2.44	1.43	1.02
512	0.84	1.63	3.12	4.13	4.68	6.38	6.47	6.16	5.48
1024	0.96	1.82	3.46	4.74	5.50	7.02	7.34	7.65	7.82
2048	1.11	2.05	4.03	5.43	6.74	7.94	8.55	9.69	9.81
4096	1.06	1.41	2.57	3.67	4.81	6.76	8.27	9.54	9.87
8192	0.99	1.44	2.73	3.91	5.06	7.01	8.88	10.09	10.47

With 12 threads, one for each core, the speedup was between 6.8 and 7.9 for n = 1024 and bigger. This is far from the perfect 12. With 24 threads, using the two threads of every core, it went up to between 7.7 (n = 1024) and 10.1 (n = 8192). With 32 threads, more than the 24 logical processors, it changed very little: from 10.09 to 10.47 at n = 8192. The small sizes are different. At n = 256 the best speedup was 2.69 with 12 threads, and with 32 threads it was 1.02, so the parallel version was not faster than the sequential one.

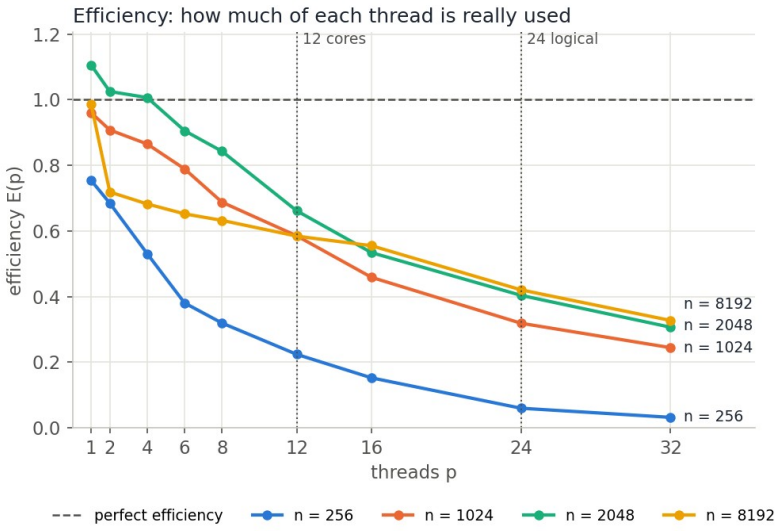


Figure 3. Efficiency E(p) = S(p) / p. The dashed line is perfect efficiency.

The efficiency in Figure 3 goes down when the threads go up, for every size. But there is a big difference between n = 2048 and the bigger sizes. With 2 threads the efficiency was 1.03 at n = 2048, but only 0.70 at n = 4096 and 0.72 at n = 8192. Some points are above 1 at n = 2048, which would be a superlinear speedup. I explain them in section 5.

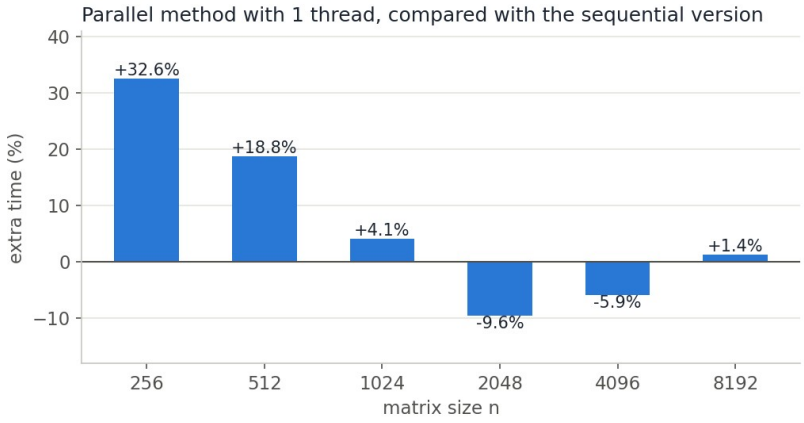


Figure 4. Extra time of the parallel version with 1 thread, compared with the sequential version. Both run the same code.

Figure 4 compares the parallel version with only one thread against the sequential version, as the assignment asks. With small matrices the parallel version was slower: 32.6% more time at n = 256 and 18.8% at n = 512. From n = 1024 the difference is between -9.6% and +4.1%, so it goes in both directions.

Table 3. Variability: the IQR of each configuration as a percentage of its median. Median IQR is the typical value for that size, and largest IQR is the worst configuration.

n	Median IQR	Largest IQR
256	10.5%	21.1%
512	5.6%	11.6%
1024	4.6%	6.8%
2048	7.0%	11.8%
4096	2.4%	5.7%
8192	2.3%	6.4%

Table 3 shows the variability. For n = 4096 and n = 8192 the runs were very stable, with a typical IQR of 2.4% and 2.3%. Very short runs changed more: at n = 256, where one multiplication takes between 0.8 and 2.8 ms, the typical IQR was 10.5%.

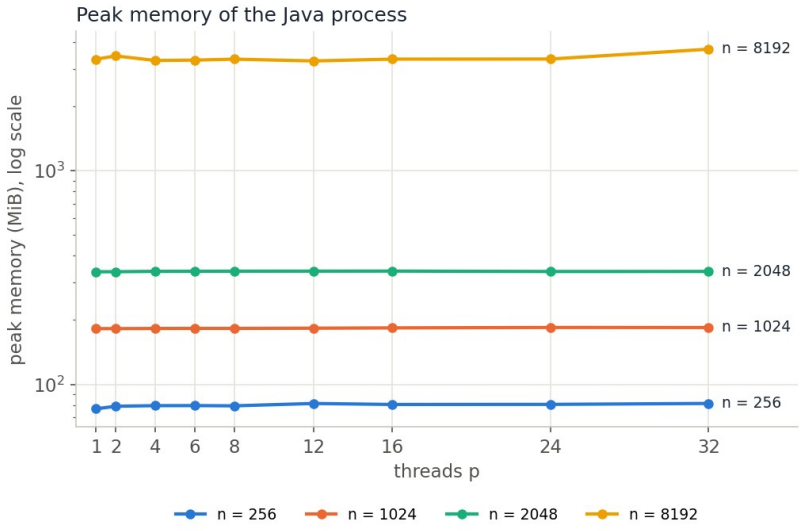


Figure 5. Peak memory of the Java process. The y axis uses a logarithmic scale.

The memory does not depend on the number of threads (Figure 5). At n = 8192 the peak was between 3,114 and 3,709 MiB for all the configurations, and at n = 2048 between 300 and 339 MiB. The threads share the same matrices, so they do not need copies. The data of A, B and C at n = 8192 is 1.5 GiB. The rest is mostly old result matrices that Java keeps until the garbage collector frees them, as I saw in Assignment 2.

The CPU use confirms that the threads really ran at the same time. At n = 8192 the Java process used 11.4 cores on average with 12 threads and 19.9 cores with 24 threads. These numbers are a bit lower than the threads because they also include the start of the program and creating the matrices, which use only one thread.

5. Discussion

The answer to my question depends a lot on the size. For n = 1024 and bigger, running in parallel helps a lot: 12 threads made the program about 7 to 8 times faster and 24 threads up to 10 times. For n = 256 it helps very little, and with many threads it is worse than doing nothing. My first and third expectations were right, but the speedup with 12 threads was lower than I expected, and the threads 13 to 24 helped more than I expected.

Small matrices do not benefit because the work is too short. At n = 256 one multiplication takes 2.1 ms. Creating a thread, starting it and waiting for it (join) also costs time, and this cost is the same for small and big matrices. With one thread it already adds 33% at n = 256 (Figure 4), and with 32 threads this cost is bigger than the time the threads save. At n = 8192 the same cost is nothing next to 112 s, so the difference with one thread is only 1.4%.

I think the drop of efficiency from n = 4096 comes from the memory. At n = 2048 each matrix uses 32 MB, so B can almost stay in the L3 cache. At n = 4096 each matrix uses 128 MB, four times more than the L3 cache of one group of cores, so the threads have to bring the numbers from the RAM. With one thread, that thread has all the RAM and the L3 cache for itself. With two or more threads, they have to share them, and each thread waits more. Another possible reason is that the CPU runs at a lower frequency when more cores are working. However, that reason would also lower the efficiency at n = 2048, and it did not, because at n = 2048 2 threads were 2.05 times faster. I did not measure cache misses or memory traffic, so I cannot prove the memory reason.

The same idea can explain why the threads 13 to 24 still helped. With 24 threads, each core runs two threads. When one of them waits for the memory, the other one can use the core. At n = 8192 this took the speedup from 7.0 to 10.1. With 32 threads there are more threads than logical processors, so Windows has to switch between them, and there was almost no more gain.

The speedup with 12 threads did not reach 12 for several reasons. One is the shared memory that I described above. Another is Amdahl's law: some parts do not run in parallel, like creating the empty matrix C and starting the threads. There is also the work at the end: the chunks are not all finished at exactly the same moment, and the last threads have to wait for the slowest one.

The superlinear points

At n = 2048 the efficiency was above 1 with 1, 2 and 4 threads (1.11, 1.03 and 1.01). That would mean that each thread is more than one full core, which is not possible here. The evidence is in Figure 4. At n = 2048 the parallel version with only one thread was already 9.6% faster than the sequential version (1.19 s against 1.32 s), and both run the same code on one core. So the sequential time of that run was a bit high. The IQR of the sequential version at n = 2048 was 0.09 s, about 7% of its median, so a difference of this size is normal variation between runs. If I use the parallel version with one thread as the reference, 2 threads are 1.85 times faster and 4 threads 3.64 times faster, below 2 and 4. So the speedup is not really superlinear.

A problem I found

My first full run had a problem. With n = 4096 the sequential version needed 37.6 s in every run, but the parallel version with only one thread needed 11 s. Both do the same work on one core, so they should take the same time. The cause was the JIT compiler. In the sequential version all the multiplication was one long call to one method, and in that run Java kept using a slower version of the code for all the 8 runs. In Assignment 2 one test had the same problem. If I had used those numbers, the parallel version would look 3.4 times faster with only one thread, which is false.

To fix it, I moved the work of each band of 64 rows into its own method. Now this method is called many times in every multiplication, in both versions, and the JIT compiles it fully after a few calls. The operations and the result did not change, and all the tests still pass. After the fix the two versions needed 11.4 s and 11.6 s at n = 4096. I stopped that first run and measured everything again with the new code, so all the results in this report come from the same code.

Limits

I used only one computer, so the numbers can change on a different CPU, especially the point where the efficiency drops. I did not set the affinity, so Windows decided where each thread ran, and with 6 threads, for example, they could be in one group of cores or in both. The times also change a bit from one run to another,

like the 10% at n = 2048. Finally, I compared only one algorithm, the blocked method, and I did not do the optional vectorisation.

6. Conclusions

Within the tested conditions, running the multiplication with threads is worth it when the matrices are big. From n = 1024, 12 threads made it about 7 to 8 times faster and 24 threads 7.7 to 10 times faster, with the same result and the same memory. More threads than logical processors did not give any real gain. For small matrices, like n = 256, the cost of the threads is too big for such a short job, and the best result was only 2.7 times faster.

The efficiency is never perfect, and it is lower when the matrices do not fit in the cache. Measuring the parallel version with one thread was very useful. It showed the cost of the threads with small matrices, and it showed that the superlinear points at n = 2048 were only variation between runs. Before that, it also helped me find the JIT problem of my first run.

Some questions remain open. I think the drop of efficiency from n = 4096 comes from the shared memory, but I would need to measure cache misses to prove it. It would also be interesting to fix the threads to one group of cores with the affinity, to try tiles of 128, and to add the vectorisation part.

References

[1] B. Goetz, T. Peierls, J. Bloch, J. Bowbeer, D. Holmes and D. Lea, Java Concurrency in Practice. Addison-Wesley, 2006.

[2] G. M. Amdahl, "Validity of the single processor approach to achieving large scale computing capabilities," in Proc. AFIPS Spring Joint Computer Conference, 1967, pp. 483-485.

[3] J. L. Hennessy and D. A. Patterson, Computer Architecture: A Quantitative Approach, 6th ed. Morgan Kaufmann, 2017.

[4] AMD, "Software Optimization Guide for AMD Family 19h Processors," pub. 56665, 2020.

[5] Oracle, "HotSpot Virtual Machine Garbage Collection Tuning Guide," Java SE 21, 2023.

Code and data: everything is in https://github.com/Pablo5250/matrix-multiplication, in the folder assignment3. There you can find the Java code, the tests, config.json with all the settings, the raw measurements (results/raw/times.csv and results/raw/runs.csv) and the scripts that make the tables and figures. run_all.bat repeats the whole study. The sequential version comes from the folder assignment2.

Use of generative AI: I used Claude (Anthropic) as a support tool to help write the code and draft the text of this report. I reviewed all the code and the text, ran every test and measurement on my own computer, and checked the results. I am responsible for the code, the data and the conclusions.